



Bibliothèques NumPy et Pandas

Mathieu RAYNAL
www.irit.fr/~Mathieu.Raynal
mathieu.raynal@irit.fr

Bibliothèque NumPy



- NumPy pour Numerical Python : <http://www.numpy.org/>
 - bibliothèque open source
 - permet d'effectuer des calculs numériques avec Python.
 - gestion facilitée des tableaux de nombres.
- Besoin d'importer le package numpy

```
import numpy as np
```

Création de tableaux

- Utilisation de la méthode array
 - crochets pour délimiter les listes d'éléments dans les tableaux.

```
a = np.array([1, 2, 3, 4])
```

- Pour créer un tableau 2D, utilisation d'une liste de listes avec des crochets imbriqués.
 - Les listes internes correspondent à des lignes du tableau.

```
b = np.array([[1, 2, 3], [4, 5, 6]])
```

Méthodes spécifiques pour créer des tableaux

- Méthodes d'initialisation
 - zeros : tableau rempli de 0
 - ones : tableau rempli de un
 - full : tableau rempli avec une valeur déterminée

Exemples	Résultat
<code>tab = np.zeros(3)</code>	<code>[0. 0. 0.]</code>
<code>tab = np.zeros(3, np.int16)</code>	<code>[0 0 0]</code>
<code>tab = np.zeros((3,2), np.bool)</code>	<code>[[False False] [False False] [False False]]</code>
<code>tab = np.ones(2, np.bool)</code>	<code>[True True]</code>
<code>tab = np.full((2, 4), 6.5)</code>	<code>[[6.5 6.5 6.5 6.5] [6.5 6.5 6.5 6.5]]</code>

Création de tableaux avec des séquences logiques

- Suite de nombre

```
np.arange(debut, fin, pas)
```

- Interpolation linéaire entre deux bornes

```
np.linspace(debut, fin, nb_val, dtype)
```

Exemples	Résultat
<code>tab = np.arange(5)</code>	<code>[0 1 2 3 4]</code>
<code>tab = np.arange(4,7)</code>	<code>[4 5 6]</code>
<code>tab = np.arange(3,20,5)</code>	<code>[3 8 13 18]</code>
<code>tab = np.linspace(3,5,4)</code>	<code>[3. 3.66666667 4.33333333 5.]</code>
<code>tab = np.linspace(3,5,4,dtype=np.int8)</code>	<code>[3 3 4 5]</code>

Génération de tableau avec valeurs aléatoires

- Pour des entiers

```
np.random.randint(borne, size=N)
```

- Pour des réels entre 0 et 1 exclu

```
np.random.random(dim)
```

Exemples	Résultat
<code>x = np.random.randint(10)</code>	Entier aléatoire entre 0 et 9
<code>tab = np.random.randint(10, size=3)</code>	Tableau de 3 entiers aléatoires entre 0 et 9
<code>mat = np.random.randint(10, size=(4,2))</code>	Matrice de 4 lignes et 2 colonnes d'entiers aléatoires entre 0 et 9
<code>x = np.random.random()</code>	Réel aléatoire entre 0 et 1 exclu
<code>tab = np.random.random(3)</code>	Tableau de 3 réels aléatoires entre 0 et 1 exclu
<code>mat = np.random.random((3,4))</code>	Matrice de 3 lignes et 4 colonnes de réels aléatoires entre 0 et 1 exclu

Utilisation de générateur de nombres aléatoires

- Classe Random Number Generator (RNG)

- Créer un générateur de nombres aléatoires ...

```
rng = np.random.default_rng ( )
```

- Ou pseudo aléatoires ...

```
rng = np.random.default_rng (42)
```

- Générer des nombres entiers aléatoires

```
rng.integers(low=, high=, size=, dtype=, endpoint=)
```

- Générer des nombres réels aléatoires entre 0 et 1 exclu

```
rng.random(size, dtype=)
```

- Générer des nombres réels aléatoires sur d'autres bornes

```
rng.uniform(min, max, nb)
```


Opérations sur des tableaux

- Modifier la forme du tableau
 - Réarranger : `np.reshape()`
 - Transposer : `np.transpose()`
 - Aplatir :
 - `np.flatten()` : créer une copie
 - `np.ravel()` : modifie juste la vue, mais c'est toujours sur le même tableau

- Combiner des tableaux
 - `np.concatenate()` : met bout à bout 2 tableaux
 - `np.vstack()` : ajouter des lignes à une matrice
 - `np.hstack()` : ajouter des colonnes à une matrice

Opérations sur des array

- Les opérations s'appliquent élément par élément
- NumPy permet de faire des opérations sans boucle Python
à Code beaucoup plus rapide.

```
a = np.array([1,2,3])
b = np.array([4,5,6])
a + b
Résultat : array([5,7,9])
```

Autres exemples :

```
a * b
a ** 2
np.sqrt(a)
np.exp(a)
```

- Permet d'appliquer des opérations entre tableaux de tailles différentes
 - NumPy étend automatiquement la dimension.

```
a = np.array([[1, 2],[3, 4]])  
b = np.array([5, 6])  
print(a+b)  
print(a*b)
```

Résultat :

```
[[ 6  8]  
 [ 8 10]]
```

```
[[ 5 12]  
 [15 24]]
```

Indexation et slicing

- Comme avec les listes Python
 - mais plus puissant.

```
a = np.array([10,20,30,40])
```

```
a[0] # 10
```

```
a[1:3] # [20,30]
```

- Pour matrices :

```
A[0,1] # élément
```

```
A[:,1] # colonne
```

```
A[1,:] # ligne
```

- Indexation conditionnelle :

```
a[a > 20]
```

à modifier une slice modifie l'array original

Fonctions mathématiques

- NumPy contient énormément de fonctions optimisées.
 - `np.sum(a)`
 - `np.mean(a)`
 - `np.max(a)`
 - `np.min(a)`
 - `np.std(a)`

- Sur les matrices :
 - `np.sum(A, axis=0)` # par colonnes
 - `np.sum(A, axis=1)` # par lignes

Bibliothèque Pandas



- Bibliothèque python pour faire de l'analyse de données
- Basé sur NumPy
- Site de référence :
https://pandas.pydata.org/pandas-docs/stable/user_guide/basics.html
- Livre de référence : *McKinney, Wes. 2012. Python for data analysis: Data wrangling with Pandas, NumPy, and IPython. " O'Reilly Media, Inc."*.

L'objet Series

- Objet à une dimension
- Extension des tableaux unidimensionnels de numpy

```
import pandas as pd  
  
tab = pd.Series( [1, 2, 3] )
```


L'objet DataFrame

- Structure à deux dimensions
- Les colonnes peuvent être de types différents
- Un DataFrame est composé de :
 - Indice de la ligne,
 - Nom de la colonne,
 - Valeur pour chaque cellule

	code_insee	nom_sans_accent	reg_nom	dep_code	academie_nom	type_commune_unite_urbaine	population	superficie_hectare
0	01001	I-bergement-clemenciat	Auvergne-Rhône-Alpes	01	Lyon	HORS UNITE URBAINE	832	1565
1	01002	I-bergement-de-varey	Auvergne-Rhône-Alpes	01	Lyon	HORS UNITE URBAINE	267	912
2	01004	amberieu-en-bugey	Auvergne-Rhône-Alpes	01	Lyon	UNITE URBAINE	14854	2448
3	01005	amberieux-en-dombes	Auvergne-Rhône-Alpes	01	Lyon	HORS UNITE URBAINE	1897	1605
4	01006	ambleon	Auvergne-Rhône-Alpes	01	Lyon	HORS UNITE URBAINE	113	602
5	01007	ambronay	Auvergne-Rhône-Alpes	01	Lyon	HORS UNITE URBAINE	2833	3359
6	01008	ambutrix	Auvergne-Rhône-Alpes	01	Lyon	UNITE URBAINE	768	518

Structure d'un DataFrame

```
df = pd.DataFrame({'A': [1, 4, 7], 'B': [2, 5, 8], 'C': [3, 6, 9]})
```

	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9

```
df = pd.DataFrame(mat,  
                  columns=["A","B","C"],  
                  index=[str((i+1)*500) for i in range (3)])
```

	A	B	C
500	1	2	3
1000	4	5	6
1500	7	8	9

- `df.index` : donne les index d'un DataFrame
- `df.columns` : donne les noms de colonnes d'un DataFrame
- `df.axes` : donne les index et les noms de colonnes d'un DataFrame
- `df.shape` : donne le nombre de lignes et colonnes
- `df.size` : donne le nombre de valeurs

Importer des données au format CSV

```
url = "https://..."
df = pd.read_csv(url)
```

- Spécifier le type

```
df = pd.read_csv("https://...", dtype={« nom_col » :str})
```

- Spécifier l'index

```
df = pd.read_csv("https://...", index_col=N)
```

Exercice

- Charger dans un DataFrame les données issues de :
<https://www.data.gouv.fr/api/1/datasets/r/f5df602b-3800-44d7-b2df-fa40a0350325>
- Définir la première colonne comme index
- Définir que chaque colonne de code contient une chaîne de caractère

Extraire des lignes

- `head()`
 - Extrait les 5 premières lignes du tableau
- `tail()`
 - Extrait les 5 dernières lignes du tableau
- `sample()`
 - Extrait aléatoirement une ligne du tableau
- Possibilité de spécifier le nombre de lignes souhaité

Indexation

- Utilisation d'index plus explicites que l'utilisation de l'indice de position
à Utilisation des noms de colonnes

- Renommer les colonnes

```
df = df.rename(columns={"ancien_nom": "nouveau_nom"})
```

```
df.columns = ["col1", "col2", "col3"]
```

- Nettoyer les noms de colonnes

```
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(" ", "_")
```

Accéder à une colonne

- `dataframe.variable`
 - Noms de colonnes sans espace ou caractères spéciaux
- `dataframe['variable']`
 - Retour sous la forme de Series
- `dataframe[['variable']]`
 - Retour sous forme de DataFrame
 - à A éviter pour une variable seule

Accéder à plusieurs colonnes

- `dataframe[['variable1', 'variable2']]`
- Privilégier l'utilisation de `loc[]`
 - `dataframe.loc[:, ['variable1', 'variable2']]`
à équivalent à `SELECT variable1, variable2 FROM dataframe`

Exercice

- Créer un DataFrame à partir des colonnes
 - code_insee
 - nom_sans_accent
 - reg_nom
 - dep_code
 - academie_nom
 - type_commune_unite_urbaine
 - Population
 - superficie_hectare
 - altitude_moyenne
 - latitude_centre
 - longitude_centre
 - niveau_equipements_services
- Afficher 10 lignes de ce nouveau tableau

Accéder à des lignes

- **df.loc[]**

- Méthode recommandée
- Fonctionne avec les labels des colonnes
- Condition sur les valeurs souhaitées pour une colonne

```
df.loc[df['superficie_hectare']>5000]
```

- Possibilité de combiner plusieurs conditions avec & et |

```
df.loc[(df['dep_code']==31.0) | (df['dep_code']==32.0)]
```

- **df.iloc[]**

- Utilise les indices.

```
df.iloc[5000]
```

Quelques fonctions bien pratiques ...

- `df['col'].min()`
- `df['col'].max()`
- `df['col'].mean()`
- `df['col'].median()`
- `df['col'].std()`
- `df.sort_values(by="col", ascending=True)`
- `df['col'].unique()` : extrait un tableau avec les différentes valeurs
- `df['col'].nunique()` : donne le nombre de valeurs différentes

Fonction groupby

- Regroupe les valeurs identiques d'une colonne en sous tableau
 - Pas exploitable directement

```
df.groupby('col')
df.groupby(['col1', 'col2'])
```

- Opérations de regroupement
 - Agrégation : min() ; max() ; mean() ; median() ; std() ; size()
 - Fonction agg() pour pouvoir agréger plusieurs colonnes

```
print(grouped.agg({
    'points': 'sum',
    'assists': 'mean'
}))
```

Exercice

- Quelle est la population de la France ?
- Quelles sont les villes de plus de 100 000 habitants ?
- Quelles sont les 3 régions avec le plus de villes de 10 000 habitants ?
 - Afficher le nom des régions et le nombre de villes de plus de 10 000 habitants

➤ Ajouter des colonnes

- Ajout de colonnes

```
df[« nom_col" ] = ...
```

Exercice

- Quelle est la ville la plus proche du centre de la France Métropolitaine ?